Document 05 — LangChain Basics

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Prerequisites

04_openai_api (Project 1 done) — you'll reuse Doc01 and Doc04 scripts again here, copied unchanged:

- `config.py` — **What:** `load_config() -> Config`. **Why:** the model choice (name, temperature) comes from config, never hardcoded. **How:** copy `01_python_foundations/practice/build_task/config.py` in as-is; don't rewrite it.
- `logging_setup.py` — **What:** `get_logger(name)`. **Why:** same consistent logging as every other document. **How:** copy `01_python_foundations/practice/build_task/logging_setup.py` in as-is; don't rewrite it. Copy `exceptions.py` (`MissingConfigError`) with it — `config.py` imports it.
- `chat_client.py` + `schemas.py` — **What:** Doc04's `get_structured_reply()`, `EXTRACT_PROMPT` and `ExtractedInfo`. **Why:** the Real-world exercise and the Build Task compare your LCEL chain against this exact raw-SDK version. **How:** copy both from `04_openai_api/practice/build_task/` into `practice/` and `practice/build_task/`, unchanged.

How to Read & Practice This Document

- **What:** building LLM calls using LangChain's tools (LCEL, prompt templates, output parsers).
- **Why:** the code you write later for real agents — including LangGraph — is built on these exact same basic pieces.
- **When:** when you need to reuse and combine many prompts or chains, not for a single one-off call (where the raw SDK is often simpler — you'll need to be able to explain which one to pick and why).
- **How to practice:** 1. Read the material once, just to get the shape of it. 2. Do the **Basic** exercise closed-book. 3. Do the **Intermediate/Real-world/Edge case** exercises with notes open. 4. Build the chain without copying Doc04's raw-SDK version — rebuild it yourself, don't paste. 5. Use **Hint 1** → **Hint 2** only after really trying. Ask for the full solution only if you say **"Show me the solution."** 6. Before moving on, explain out loud when you'd pick LangChain over the raw SDK for a new task, and why. If you can't, you're not done.

Jump to: Learn It in Order (start here) · Core Concepts · Practice Exercises · Build Task

The Story — what this document is actually building

Picture this: in Doc04 you built a chatbot by hand — building a `messages` list yourself, calling the OpenAI API yourself, reading the reply yourself, every single time. That worked, but every new feature meant rewriting the same plumbing again from scratch.

This document is about a library, LangChain, that takes exactly that repeated plumbing and turns it into small, swappable pieces you connect instead of rewrite. **First**, `ChatOpenAI` replaces your raw API call — same model, same underlying request, just a standard shape that works the same way no matter which AI provider you're actually using underneath. **Second**, a prompt template separates the fixed wording of a prompt from the parts that change each time, so the prompt itself becomes something you can log, test, and swap out — not a string you build by hand with f-strings scattered through your code. **Third**, LCEL's `|` symbol lets you snap a prompt template, a model, and an output parser together into one pipeline — `prompt | model | parser` — so calling it once runs all three steps in order, and the whole thing behaves like one reusable object. **Fourth**, the output parser is what turns the model's raw text reply into a real Python object your code can actually use, and it checks that shape *after* the model answers — a different method from Doc04's structured output, which shapes the answer *while* it's being generated.

That's the whole story: each Core Concept below is one interchangeable piece, and LCEL is the glue that snaps them together. The Build Task at the end asks you to build exactly that — one small, reusable chain module — and prove, side by side, that it gives the same answer as Doc04's raw version, so you can actually explain, not just guess, when the extra layer is worth its cost.

Learn It in Order — One Chain, Built Step by Step

New to all of this? Start here. This is the whole document in one place: we build **one** small chain, adding **one** new piece at a time, and run it after every step so you *see* what changed. Type each step yourself and run it. Once this clicks, the detailed Core Concepts below are just the same five pieces explained more deeply — and the Practice Exercises are these exact steps, in this exact order.

> Setup first (once): `pip install langchain langchain-openai langchain-core`, and make sure your `OPENAI_API_KEY` is in `.env` (same key as Doc04). Every snippet below is a complete, runnable file.

Step 0 — How you did it in Doc04 (the thing we're replacing)

In Doc04 you called the model by hand, every time:

```
from openai import OpenAI
client = OpenAI()
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Say hello in one word."}],
)
print(response.choices[0].message.content)   # -> Hello
```

That works. But the `messages` list, the `create(...)` call, and digging the text out of `.choices[0].message.content` get rewritten in every script. The next five steps replace each of those with one reusable piece.

Step 1 — `ChatOpenAI`: the same call, a simpler shape

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o-mini", temperature=0, timeout=30)
reply = model.invoke("Say hello in one word.")
print(reply.content)      # -> Hello
```

What's new: `.invoke(...)` instead of `client.chat.completions.create(...)`, and you read `reply.content` instead of `.choices[0].message.content`. Same model, same answer — just a shorter, standard shape. (`reply` is an `AIMessage`; always read `.content` for the text.)

Step 2 — Prompt template: write the fixed part once

Most prompts have a fixed part and a changing part. A **template** keeps them separate — the fixed words written once, the changing part as a `{slot}`:

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template("Translate to French: {text}")

messages = prompt.invoke({"text": "good morning"})   # fills the slot only
print(messages)                                      # no API call, no cost yet

reply = model.invoke(messages)                       # now call the model
print(reply.content)      # -> Bonjour
```

What's new: `prompt.invoke({...})` only fills the blanks and returns messages — it does **not** call the model (so it's free to test). You still pass those messages to `model.invoke(...)` yourself. Step 3 removes that hand-off.

Step 3 — LCEL `|`: snap the pieces into one chain

The `|` symbol connects pieces: it runs the left one, then feeds its output into the right one. So `prompt | model` is "fill the slots, then call the model" as a single object:

```
chain = prompt | model
reply = chain.invoke({"text": "thank you"})
print(reply.content)      # -> Merci
```

What's new: one `chain.invoke({...})` now does both steps. The chain behaves like one piece — you can pass it around and reuse it everywhere.

Step 4 — Output parser: get clean text out of the end

Right now `chain.invoke(...)` still returns an `AIMessage`, so you keep writing `.content`. Add a **parser** as the last link to get plain text straight out:

```
from langchain_core.output_parsers import StrOutputParser

chain = prompt | model | StrOutputParser()
print(chain.invoke({"text": "please"}))   # -> S'il vous plaît   (plain text)
```

What's new: `StrOutputParser()` is the last step; now the chain hands you the text directly, no `.content` needed. This is the full three-piece shape — **`prompt | model | parser`** — that every LangChain chain is made of.

Step 5 — A typed object instead of text

Often you don't want text — you want fields your code can use. Ask the model to fill a Pydantic shape with `.with_structured_output(...)`, and you get a real object back:

```
from pydantic import BaseModel

class Person(BaseModel):
    name: str
    age: int

extractor = model.with_structured_output(Person)
person = extractor.invoke("Sara Khan is 30 years old.")
print(person.name, person.age)   # -> Sara Khan 30
```

What's new: the reply is a `Person` object (`person.name`, `person.age`), not text you have to parse yourself. This is the same "structured output" idea from Doc04, now one method call.

Going further (advanced). Steps 1-5 are everything you *need*. The next five steps are the pieces you'll reach for on real pipelines — a dynamic prompt, the different parser types, and the Runnables that let a chain branch, run in parallel, and route. Same running example, still one piece at a time.

Step 6 — Dynamic prompts: system rules + chat history

A `from_template` prompt is **static** — one fixed message. A real chat needs a **dynamic** prompt: fixed system rules, the user's turn, and the past conversation. `from_messages` builds that, and `MessagesPlaceholder` holds a whole history:

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import HumanMessage

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful {role}."),
    MessagesPlaceholder("history"),
    ("human", "{question}"),
])
messages = prompt.invoke({
    "role": "tutor",
    "history": [HumanMessage(content="My name is Sara.")],
    "question": "What is my name?",
})
print([m.type for m in messages.to_messages()])   # ['system','human','human']
```

What's new: `from_messages` keeps the system rules separate from the user's text (safer — see Project 9), and `MessagesPlaceholder` drops the whole past chat into one slot. Static prompt = `from_template`; dynamic = `from_messages`.

Step 7 — The four kinds of output parser

You've used `StrOutputParser` (text) and `.with_structured_output` (typed). In full, there are four ways to shape the output, from loosest to strictest:

```
from langchain_core.output_parsers import JsonOutputParser

# JSON parser: a loose dict, good for quick tools
json_chain = (
    ChatPromptTemplate.from_template(
        "Reply with JSON keys city and country for: {place}")
    | model | JsonOutputParser()
)
```

```
print(json_chain.invoke({"place": "the Eiffel Tower"}))
# -> {'city': 'Paris', 'country': 'France'}   (a real dict)
```

What's new — pick by how strict you need to be:

- `StrOutputParser` → plain text (`str`).
- `JsonOutputParser` → a `dict` , no shape checking (quick, loose).
- `PydanticOutputParser` → checks a shape **after** the model answers; raises `OutputParserException` if it's wrong (works with any model).
- `.with_structured_output(Model)` → shapes it **during** generation; fewest errors, needs a provider that supports it (Step 5).

Step 8 — Runnables: put your own function in the chain

Every piece you've connected with `|` is a **Runnable** (anything with `.invoke`). That means your *own* function can be a step too — wrap it in `RunnableLambda` :

```
from langchain_core.runnables import RunnableLambda

shout = RunnableLambda(lambda text: text.upper())
chain = prompt_fr | model | StrOutputParser() | shout
print(chain.invoke({"text": "thank you"}))   # -> MERCI
```

What's new: `RunnableLambda` turns any function into a chain step, so you can reshape data between the model and the next piece — no special LangChain class needed.

Step 9 — Run things in parallel, and keep the input

A `dict` of Runnables (`RunnableParallel`) runs its branches **at the same time** and returns a dict of results; `RunnablePassthrough` carries the original input forward unchanged:

```
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

french = ChatPromptTemplate.from_template("French for: {text}") | model \
    | StrOutputParser()
pipeline = RunnableParallel(french=french, original=RunnablePassthrough())
print(pipeline.invoke({"text": "hello"}))
# -> {'french': 'Bonjour', 'original': {'text': 'hello'}}
```

What's new: parallel branches are faster than one-after-another, and `RunnablePassthrough` is how RAG (Doc08) keeps the user's question next to the retrieved text.

Step 10 — Conditional chains: route by the input (`RunnableBranch`)

The three chain shapes are **sequential** (`|`), **parallel** (Step 9), and **conditional** — pick a different chain based on the input. `RunnableBranch` is an if / elif / else inside a pipeline:

```
from langchain_core.runnables import RunnableBranch

to_fr = ChatPromptTemplate.from_template("French for: {text}") | model \
    | StrOutputParser()
to_es = ChatPromptTemplate.from_template("Spanish for: {text}") | model \
    | StrOutputParser()
echo = ChatPromptTemplate.from_template("Repeat: {text}") | model \
    | StrOutputParser()

router = RunnableBranch(
    (lambda x: x["lang"] == "fr", to_fr),
    (lambda x: x["lang"] == "es", to_es),
    echo,                              # default, when nothing matched
)
print(router.invoke({"lang": "fr", "text": "good night"}))   # -> Bonne nuit
```

What's new: `RunnableBranch` routes to one of several chains by a condition — the LangChain way to branch. This is the shape a real support pipeline uses to send each ticket to the right handler (Project 6).

You've now seen it, zero to advanced

The basics (Steps 1-5) and the advanced pieces (Steps 6-10) together are the whole of LangChain's fundamentals. Everything later (agents, RAG, LangGraph) is built from exactly these. Now do the Practice Exercises **in this same order** — each one is a step you just read, as a real, meaningful task:

You read	Now practice
Steps 1-4 (the three-piece chain)	Basic — your first LCEL chain
Step 5 (typed output)	Intermediate — swap in a structured parser
Step 6 (dynamic prompts, system + human)	Real-world — rebuild Doc04's extraction
Steps 1-7 together, vs. Doc04	Real-world — rebuild Doc04's extraction
Steps 8-10 (Runnable types + routing)	Real-world — a support router
Chain powers (parallel, fallback, stream)	Real-world (part 2)
When a template slot is wrong	Edge cases
Step 7 — when a parser fails	Failure

The detailed Core Concepts below explain each piece more deeply (settings, traps, and when to use which) — read them after you've run the steps above and want the full picture.

Core Concepts (read this first — everything you need is here)

Topics on this page: What a library gives you, and what it costs · `ChatOpenAI` · Prompt templates · Few-shot prompting · LCEL · Output parsers · The three LangChain packages

What a library gives you, and what it costs

A **library** is code someone else already wrote and tested, so you do not write it again. In Doc04 you built every request by hand: a `messages` list, `client.chat.completions.create(...)`, then reading `.choices[0].message.content` yourself, every single time. LangChain (`prompt | model | parser`) gives you that same shape as three interchangeable pieces you connect once. It is a toolbox, not a different way of thinking about the model — the model call underneath is unchanged.

How it really works

- LangChain adds one layer between your code and the real API call. A bug can now be in your code, in the library's code, or in the API — one more place to look.
- LangChain 1.0 (Oct 2025) reorganized the package: old classes (`LLMChain`, `AgentExecutor`) moved out. A 2023 tutorial's imports will not match what you install today — check the date before copying code.
- The saving only shows up at scale: one script gains nothing from the wrapper. Ten scripts sharing one `ChatOpenAI` object and one parser style gain a single place to fix a bug or swap a model.
- Every `ChatOpenAI().invoke()` call is still, underneath, the same HTTP POST Doc02 taught you to wrap in a timeout and retry logic — LangChain just builds that request for you.

Situation	What to do	Why
One quick, one-off call in a small script	Raw OpenAI SDK (Doc04)	Fewer layers, easier to debug
Many similar prompts (extract, summarize, classify)	LangChain	One shared shape; less copied code
You may switch model provider later	LangChain	Every chat model shares `.invoke()`
Building agents, RAG, or LangGraph workflows	LangChain	Doc06-Doc11 are built on these pieces
A brand-new API feature, day one	Raw SDK for that call	The wrapper may not support it yet

Common mistakes:

- *Mistake:* reaching for LangChain on a one-line nightly script. → *Symptom:* a five-line task now has more imports than logic. → *Fix:* if you are not reusing the prompt or chain elsewhere, use the raw SDK.
- *Mistake:* copying LangChain code from an old tutorial without checking the version. → *Symptom:* `ImportError: cannot import name 'LLMChain'`. → *Fix:* rebuild with LCEL (`prompt | model | parser`) — see the packages topic below.

Where you'll meet it: this choice returns in Doc06 and Doc07 (tools and agents are LangChain-native), and in Doc08 and Doc09, which are built on these same pieces. Project 6 is entirely about knowing when LCEL is worth it and when it is not.

Quick cheat sheet:

- A library saves repeated work; it costs one more layer to debug.
- One-off call → raw SDK (Doc04). Many similar prompts, agents, RAG → LangChain.
- Check the date on any LangChain tutorial — 1.0 renamed and removed a lot.
- You must be able to explain your choice, not just make it.

`ChatOpenAI` : a wrapper, not a different model

`ChatOpenAI` sends the exact same request to the exact same model Doc04 already taught you to call — only the Python shape changes. Instead of `client.chat.completions.create(...)` you call `.invoke(messages)` ; instead of a raw `response` you get an `AIMessage` with `.content` for the text. The API key, `.env` loading, and rate limits are still Doc04's rules — nothing about talking to OpenAI itself is new here.

How it really works

- LangChain gives every provider's chat model (OpenAI, Anthropic, Google, local) the same interface — `.invoke()` , `.stream()` , `.batch()` , the same message types. Code written for `ChatOpenAI` mostly still works if you swap in `ChatAnthropic` .
- `max_retries` here is Doc02's backoff loop, built in — LangChain retries and waits between attempts for you. Keep it small (2 is plenty); `.with_retry()` on a whole chain (see LCEL, below) is the other place retries can live, so pick one layer, not both.
- `timeout` is Doc02's rule again: never call a model with none. LLM calls need a long read timeout (the model may think 30-60+ seconds) — the same connect/read split Doc02 drew.
- `reply.usage_metadata` gives token counts the same way Doc04's `response.usage` did — same numbers, a different attribute name. Log them with Doc01's `get_logger` , and Doc03's cost formula still applies unchanged.
- `.stream()` yields `AIMessageChunk` pieces you can add together with `+` — the same chunked-reply idea as Doc04's `delta.content` , except LangChain joins the pieces for you instead of you guarding every chunk with `or ""` .

- Errors raised through `ChatOpenAI` are the same named exceptions Doc04 taught — `RateLimitError`, `AuthenticationError`, `APITimeoutError` — LangChain does not invent new ones. Catch them the same way: permanent (401/400) don't retry, temporary (429/timeout/5xx) retry with backoff (Doc02's groups).
- Printing `reply` instead of `reply.content` dumps the whole `AIMessage` object (`content='...' response_metadata={...}`) — an easy, harmless-looking bug.

Setting	What it does	When to set it
`model`	Which model to call	Always — from config (Doc01), never hardcoded
`temperature`	How random the answer is	`0` for extraction/classification, higher for creative writing
`timeout`	Max seconds to wait	Always — Doc02's rule, no exceptions
`max_retries`	Retries on failure	Small (`2`); one retry layer only
`api_key`	Which key to use	Usually leave out — reads `OPENAI_API_KEY`, same as Doc04

Common mistakes:

- *Mistake:* thinking `ChatOpenAI` is a smarter or different model. → *Symptom:* a wrong answer with the raw SDK is still wrong here — fixing the wrapper changes nothing. → *Fix:* fix the prompt or the model choice, not the wrapper.
- *Mistake:* `print(reply)` instead of `print(reply.content)`. → *Symptom:* your UI or log shows `content='...' response_metadata=...` instead of clean text. → *Fix:* always read `.content` for the text.

Where you'll meet it: every later document that calls a model. Doc06 attaches tools with `.bind_tools(...)`; Doc07 passes it to `create_agent(...)`; Doc11 often gives each agent its own `ChatOpenAI` — a cheap model for routing, a stronger one for writing.

Quick cheat sheet:

- Same OpenAI model as Doc04 — only the Python shape is new.
- `.invoke()` returns an `AIMessage` — use `.content` for the text.
- Always set `model` (from config), `timeout`, and a small `max_retries`.
- `.stream()` for live typing, `.batch()` for many inputs at once.
- One retry layer: either `max_retries` here, or `.with_retry()` on the chain — not both.

Prompt templates: keeping the fixed part separate from the changing part

A **prompt template** is a prompt with empty slots — fixed wording written once, changing parts named in curly braces like `{text}`, filled in each call. Think of it like a form letter: the boilerplate is

printed once, only the blanks change per recipient.
`ChatPromptTemplate.from_template("Extract the {field} from: {text}")` replaces building the string by hand with f-strings scattered through your code, and turns the prompt into a real object you can log, version, and test without spending a token.

How it really works

- `.from_template(...)` builds one human message; `.from_messages([...])` builds a full list, keeping system rules separate from user input — the same separation Doc04 taught you to keep between the `system` message and the `user` turns.
- `MessagesPlaceholder("history")` drops a whole list of past messages into one slot — this is how a chat prompt carries Doc03's "no memory, resend everything" rule.
- `prompt.invoke({...})` only fills slots and returns messages — no API call, no cost — so you can test a prompt for free before it ever reaches the model.
- A missing slot raises `KeyError` at build time, before any call goes out — cheaper than sending a broken prompt and puzzling over a strange answer three functions later.
- Extra keys you pass but the template doesn't use are silently ignored, so a typo'd key (`"qestion"`) shows up as a *missing* variable, not an extra one.
- Real curly braces in the text (a JSON example) must be doubled — `{{` and `}}` — or LangChain reads them as slot names and raises a confusing "missing variable" error.
- `.partial(...)` pre-fills one slot (commonly a parser's format instructions) so the caller only supplies the rest.

Situation	What to do	Why
One simple message with slots	`from_template(...)`	Shortest; becomes one human message
System rules + user input	`from_messages([...])`	Keeps rules apart from what changes
Chat history in the prompt	`MessagesPlaceholder("history")`	Carries Doc03's resend-everything memory
JSON example inside the text	Double the braces: `{{ }}`	Single braces are read as a slot
Same template reused everywhere	Keep it in `prompts.py`, import it	One place to change and review

Common mistakes:

- *Mistake:* writing a JSON example with single braces, like `Reply as {"name": "..."}`. → *Symptom:* a confusing error naming a "missing variable" that is actually part of your JSON. → *Fix:* double every real brace — `{{` and `}}`.

- *Mistake:* renaming a template variable but not the code that fills it. → *Symptom:* `KeyError` at `.invoke()`, or a typo'd key silently treated as missing. → *Fix:* keep the template and its caller close, or import the exact slot names from one place.

Where you'll meet it: this document's Build Task (`prompts.py`). Doc08 uses a template with `{context}` and `{question}` slots to answer from company documents. In Doc11 each agent has its own named system prompt, easy to see and change. Project 9 is built on why user text must go into a slot and never get mixed into the system rules.

Quick cheat sheet:

- Fixed words in the template; changing parts in `{slots}`.
- `from_template` for one message; `from_messages` for system + human.
- `MessagesPlaceholder("history")` for chat history.
- Real braces in the text → double them: `{{` `}}`.
- `prompt.invoke({...})` fills slots only — free, no API call.

Few-shot prompting: teaching the model by example

Some tasks are hard to *describe* but easy to *show*. **Few-shot prompting** puts a handful of example input→output pairs right in the prompt, so the model copies the pattern instead of guessing from a description. ("Zero-shot" is no examples; "few-shot" is a few.) It's the cheapest way to pin down a fuzzy category, an exact output format, or a house style — before you reach for anything heavier like fine-tuning.

How it really works

- The raw way is just messages: put example turns — `("human", example_input)`, `("ai", example_output)` — into the list *before* the real question. The model treats them as "this is how it's done here."
- The reusable way is `FewShotChatMessagePromptTemplate(examples, example_prompt)`: it expands a list of example `dict`s into those human/ai pairs, and you drop it into a `ChatPromptTemplate` between the system rules and the real input.
- Examples are **re-sent on every call** — they cost input tokens each time (Doc03's cost rule). Keep them few (2-5) and representative; more is rarely better and sometimes worse.
- Biased or lopsided examples bias the output — five positive examples and the model leans positive. Cover the real range, including an edge case or two.
- Few-shot and structured output stack: show examples of the *shape* you want, and still validate with a parser (the model can copy a format and still get a value wrong).

```
from langchain_core.prompts import (ChatPromptTemplate,
    FewShotChatMessagePromptTemplate)

examples = [
```

```
    {"text": "I love it!", "label": "positive"},
    {"text": "worst ever", "label": "negative"},
]
example_prompt = ChatPromptTemplate.from_messages(
    [("human", "{text}"), ("ai", "{label}")])
few_shot = FewShotChatMessagePromptTemplate(
    examples=examples, example_prompt=example_prompt)

prompt = ChatPromptTemplate.from_messages([
    ("system", "Classify the sentiment as positive or negative."),
    few_shot,                          # expands into the 2 example pairs
    ("human", "{text}"),
])
# prompt.invoke({"text": "this is great"}) -> system, 2 examples, then the human
```

Situation	What to do	Why
A fuzzy category the model keeps missing	2-5 labelled examples	Shows the boundary words can't
An exact output format or style	Examples of that format	The model copies it reliably
A clear instruction already works	No examples (zero-shot)	Examples cost tokens every call
The task needs current/private data	RAG (Doc08), not examples	Few-shot teaches *pattern*, not *facts*

Common mistakes:

- *Mistake:* 10+ examples "to be safe". → *Symptom:* every call is expensive and slower, with no better answers. → *Fix:* 2-5 representative examples; add one only if it fixes a real miss.
- *Mistake:* all examples of one kind. → *Symptom:* the model leans that way on ambiguous input. → *Fix:* balance the examples across the real range.

Where you'll meet it: the Intermediate exercise's classification is a natural place to add two examples and watch accuracy jump. Doc11's agents often carry a couple of examples in their system prompt so each specialist stays on-style.

Quick cheat sheet:

- Few-shot = show examples; zero-shot = describe only.
- Raw: `("human", in)`, `("ai", out)` pairs before the real turn.
- LangChain: `FewShotChatMessagePromptTemplate(examples, example_prompt)`.
- 2-5 balanced examples; they cost tokens every call.
- Teaches a *pattern*, not *facts* — for facts, use RAG.

LCEL: chaining pieces together with `|`

LCEL (LangChain Expression Language) connects pieces with the `|` symbol: `chain = prompt | model | parser`. Calling `chain.invoke(inputs)` runs each step in order, feeding one step's output straight into the next. Every piece — and the whole chain — is a `Runnable`: anything with `.invoke()`, `.stream()`, `.batch()`, and async versions (Doc08b). So a three-step chain behaves exactly like one single step: you can pass it around, nest it inside a bigger chain, and bolt on retries in one line, instead of writing the same glue code for every new chain.

How it really works

- `a | b` makes a new `Runnable` that runs `a`, then hands its result to `b` — no hidden magic beyond that; every piece takes one input and returns one output.
- A plain Python function becomes a Runnable step automatically; a `dict` becomes a step that runs each value in parallel and returns a dict of results (`RunnableParallel`).
- `RunnablePassthrough` keeps the original input alive for a later step — the standard way RAG (Doc08) keeps the user's question next to the retrieved context.
- `.with_retry(stop_after_attempt=3)` is Doc02's backoff loop, built into the chain itself — LCEL does natively what Doc02 built by hand with a `for` loop and `time.sleep`. Pick exactly one retry layer: the chain's `.with_retry()`, the model's `max_retries`, or your own loop — never two, same rule Doc02 and Doc04 both taught.
- `.with_fallbacks([backup_chain])` swaps in a second chain — a cheaper model, a different prompt — when the first one raises. One layer above a single retry: retry the same thing again, fall back to something different.
- A broken `|` connection (`model | prompt` — the prompt expects a `dict`, the model returns an `AIMessage`) fails deep inside LangChain with a confusing type error. Debug by running each piece alone: `prompt.invoke(inputs)`, then `model.invoke(that_result)`.

Situation	What to do	Why
Steps run one after another	`a | b | c`	Simple, readable pipeline
Your own code in the middle	A function, or `RunnableLambda`	Reshape data between steps
Two independent tasks, same input	`RunnableParallel` (or a `dict`)	Run at the same time — faster
Keep the original input for later	`RunnablePassthrough`	A later step still needs the first input
API fails for a moment	`.with_retry(...)`	Doc02's backoff, built in — one retry layer only
Main model down or too slow	`.with_fallbacks([...])`	Backup chain instead of a failed call

Common mistakes:

- *Mistake:* connecting two pieces whose shapes don't match, like `model | prompt`. → *Symptom:* a confusing type error deep inside LangChain, mentioning `AIMessage` or `dict` at a step boundary. → *Fix:* for each `|`, check what the left side returns and what the right side accepts; run pieces alone to isolate it.
- *Mistake:* `.with_retry()` on the chain **and** `max_retries` on the model **and** your own loop. → *Symptom:* one real failure turns into far more calls than your logs show — Doc02's retry-stacking mistake, one layer higher. → *Fix:* pick exactly one retry layer.

Where you'll meet it: this document's Build Task (`build_extraction_chain() -> Runnable`). Project 6 is entirely LCEL patterns — parallel steps, retry, and fallbacks in a support pipeline. Doc08's RAG chain is `{"context": retriever, "question": RunnablePassthrough()} | prompt | model | parser`. Doc09 and Doc11's graphs are Runnables too, called with the same `.invoke()` you learn here.

Quick cheat sheet:

- `a | b` = run `a`, hand its output to `b`. Nothing more.
- Every chain is a `Runnable`: `.invoke()`, `.stream()`, `.batch()`, `.ainvoke()`.
- `RunnableParallel` / `dict` for parallel steps; `RunnablePassthrough` to keep the input.
- `.with_retry()` = Doc02's backoff, built in. `.with_fallbacks()` = a backup chain.
- One retry layer only — chain, model, or your own loop, never two.

Output parsers: turning raw text into a real Python object

An **output parser** is the last step of a chain — it turns the model's reply into a real Python value your code can use: a `str`, a `dict`, or a Pydantic object. `StrOutputParser` pulls out the text. `PydanticOutputParser` checks the reply against a Pydantic model and raises a named `OutputParserException` when it doesn't match. This is the same problem Doc02's JSON topic taught you about raw HTTP responses — text that parses fine but is still the wrong shape — just showing up in an LLM's reply instead of an API's.

How it really works

- Two different methods, and you must know which one a chain uses. **Check after**: the prompt asks for a shape in words, the model writes free text, the parser checks it afterward — works with any model, but the model can still get the shape wrong. **Shape during**: `model.with_structured_output(Model)` uses the provider's constrained decoding — the exact mechanism Doc04's structured-output topic explained — so the shape is enforced while the model writes, and LangChain hands you the parsed object directly.
- `PydanticOutputParser` needs `parser.get_format_instructions()` inserted into the prompt (usually via `.partial(...)`), or the model never learns what shape to write.

- A parser failure is `OutputParserException` (Pydantic's `ValidationError` underneath) — catch it near the call with a clear message, the same discipline Doc04 taught for a refusal or `LengthFinishReasonError`, never a bare `except`.
- Right shape is not the same as right values — a parser lets `{"price": 0}` through because `0` is a valid `int`. Add a Pydantic `field_validator` for business rules the schema can't express, same fix Doc04's structured-output topic gave.
- A mismatched prompt and parser (prompt asks for tags, chain ends in an `Order` parser) fails on every call, or — with a loose `JsonOutputParser` — quietly returns the wrong type instead of erroring.

Situation	What to do	Why
Only need the text	`StrOutputParser()`	`AIMessage` → plain `str`
Typed object, model supports it	`model.with_structured_output(Model)`	Doc04's constrained decoding — fewest shape errors
No structured-output support	`PydanticOutputParser(pydantic_object=Model)`	Works with any text model; checks after
A loose, quick `dict`	`JsonOutputParser()`	Less strict, good for quick tools
Shape can fail, user is waiting	Catch `OutputParserException` / `ValidationError`	Show a clean message, don't crash

Common mistakes:

- *Mistake:* prompt and parser describing different shapes — prompt says "list the tags," chain ends in an `Order` parser. → *Symptom:* every call fails, or a loose parser quietly returns the wrong type. → *Fix:* make the prompt's format instructions and the parser's model describe the same shape.
- *Mistake:* treating a parsed object as a correct one. → *Symptom:* `{"price": 0}` passes because `0` is a valid `int`, but it's still wrong. → *Fix:* add a Pydantic `field_validator` — shape is guaranteed, truth is not (Doc04).

Where you'll meet it: this document's Intermediate and Failure exercises, and the Build Task, compared against Doc04's raw structured output. Doc06's tool arguments use the same Pydantic check. In Doc11 and Project 4, agents hand each other typed objects (a reviewer's `approved: bool`); a parser failure here is the handover point breaking — Doc02's "valid JSON, wrong shape" problem, one layer up.

Quick cheat sheet:

- Model gives text; parser gives a Python value.

- `StrOutputParser` → `str`. `PydanticOutputParser` / `with_structured_output` → typed object.
- "Check after" can fail — catch `OutputParserException`.
- `with_structured_output` = Doc04's constrained decoding, same guarantee.
- Shape is not truth — add validators for values the schema can't check.

The three LangChain packages: `langchain-core`, `langchain-openai`, `langchain`

LangChain is not one package — it splits into `langchain-core`, `langchain-openai` (or another provider), and `langchain`, each installed separately with `pip` but sharing one naming quirk: install with a dash (`pip install langchain-core`), import with an underscore (`from langchain_core import ...`). Splitting it this way means a small extraction service installs only what it needs, and provider-specific code updates without touching the core — the same reason Doc01 taught pinned, minimal `requirements.txt` files.

How it really works

- `langchain-core`: the building blocks — `Runnable`, messages, prompt templates, output parsers, `@tool`. Very few dependencies.
- `langchain-openai` (and siblings `langchain-anthropic`, `langchain-google-genai`): provider-only pieces, `ChatOpenAI` / `OpenAIEmbeddings`. Swapping providers means changing this one import, not your prompts or parsers.
- `langchain`: high-level tools built on the other two — `create_agent` (built on LangGraph), `init_chat_model`. `langgraph` is its own separate package again.
- LangChain 1.0 (October 2025) shrank `langchain` down to agents and a few helpers. Old classes (`LLMChain`, `RetrievalQA`, `AgentExecutor`) moved to `langchain-classic`. A tutorial importing `from langchain.chains import LLMChain` predates this — rebuild with LCEL (`prompt | model | parser`) or `create_agent` instead.
- Mixed major versions across the `langchain-*` packages (core 1.x with openai 0.1.x) produce `ImportError`s that look like bugs in your own code. Check with `pip list | grep langchain` — all packages should share a major version — and pin them together in `requirements.txt`, the same Doc01 pinning discipline.

Situation	What to do	Why
Chains with prompts, models, parsers	`langchain-core` + `langchain-openai`	All LCEL needs
Building agents (Doc06/Doc07)	Also install `langchain`	`create_agent` lives there
Building graphs (Doc09)	Also install `langgraph`	Its own package
A vector store or loader (Doc08)	Install that one integration package	Keeps other integrations out

Situation	What to do	Why
Old code imports `LLMChain` / `AgentExecutor`	Rewrite with LCEL / `create_agent`	Removed from `langchain` 1.0

Common mistakes:

- *Mistake:* upgrading only one `langchain-*` package. → *Symptom:* strange `ImportError`s or missing attributes that look like your own bug. → *Fix:* `pip list | grep langchain` — upgrade and pin them together.
- *Mistake:* copying an old tutorial's `from langchain.chains import LLMChain`. → *Symptom:* `ImportError: cannot import name 'LLMChain'` on a fresh install. → *Fix:* rebuild with LCEL, or `create_agent` for agents — check the tutorial's date first.

Where you'll meet it: every `requirements.txt` from here on, and the top of every file in Doc06-Doc11 and the projects. Project 11 and Project 13's real code mixes imports from all three packages plus `langgraph` in one file — each import line tells you which layer that piece comes from.

Quick cheat sheet:

- `langchain-core` = building blocks. `langchain-openai` = provider. `langchain` = agents/helpers. `langgraph` = graphs.
- Install with a dash, import with an underscore.
- `LLMChain` / `AgentExecutor` are gone from 1.0 — use LCEL and `create_agent`.
- Upgrade and pin all `langchain-*` packages together.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- LangChain — Introduction — what LangChain is, and isn't.
- LangChain — Concepts hub — start with chat models, prompt templates, LCEL, output parsers.
- LangChain Academy — a free official course; do the intro alongside this document.

Practice Exercises

Setup: same venv as before — if it's not active, `source ../01_python_foundations/.venv/bin/activate` (or your own venv for this folder). New package for this document: `pip install langchain langchain-openai langchain-core`.

Where your code lives: all of it under `05_langchain_fundamentals/practice/` (`mkdir -p practice`), never loose beside this README — same convention as Doc01, Doc02, and Doc04.

Exercises are grouped **by topic, not by level**.

The full file layout, all exercises:

```
practice/
├── lcel_chain_basics_practice.py    Basic
├── config.py, exceptions.py         copied from Doc01, unchanged (Basic)
├── structured_output_practice.py    Intermediate + Failure (two sections)
├── lcel_vs_raw_sdk_practice.py      Real-world
├── lcel_patterns_practice.py        Real-world (part 2)
├── runnable_routing_practice.py     Real-world (support router)
├── prompt_template_practice.py      Edge cases
├── chat_client.py                   copied from Doc04, unchanged
├── schemas.py                       copied from Doc04, unchanged
└── build_task/                      Build Task — its own folder
    ├── chain.py                     build_extraction_chain() -> Runnable
    ├── prompts.py                   get_extraction_prompt()
    ├── compare_with_raw_sdk.py      the 3 Test Cases
    ├── chat_client.py               copied from Doc04, unchanged
    ├── schemas.py                   copied from Doc04, unchanged
    ├── config.py                    copied from Doc01, unchanged
    ├── exceptions.py                copied from Doc01, unchanged
    └── logging_setup.py             copied from Doc01, unchanged
```

Why each script exists:

- `lcel_chain_basics_practice.py` — the 3-piece chain every LangChain thing you'll build is made of.
- `structured_output_practice.py` — feels, directly, the difference between "shape during" (`.with_structured_output`) and "check after" (`PydanticOutputParser`), and which error each one raises.
- `chat_client.py` / `schemas.py` — Doc04's raw extraction, copied unchanged, so the Real-world exercise has a real version to compare against.
- `lcel_vs_raw_sdk_practice.py` — the only way to actually compare LangChain vs. raw SDK is having both versions side by side.
- `lcel_patterns_practice.py` — runs steps in parallel, falls back to a backup, and streams, once each, so none of them is new when Project 6 uses them.
- `runnable_routing_practice.py` — a real routing pipeline: `RunnableBranch` sends each ticket to the matching reply chain, `RunnableLambda` post-processes — the exact shape Project 6 and every production LCEL pipeline use.
- `prompt_template_practice.py` — knowing whether a broken template fails while building the prompt or only once the call goes out changes how fast you can debug it.
- `build_task/chain.py` / `build_task/prompts.py` — the one chain module every later document imports instead of writing raw SDK calls again.

- `build_task/compare_with_raw_sdk.py` — the Build Task's test file: proves the LCEL version and Doc04's raw version agree, and checks the two failure cases.
- `build_task/config.py` / `exceptions.py` / `logging_setup.py` — copied unchanged from Doc01, same settings/logging behavior as every other document.
- `build_task/chat_client.py` / `schemas.py` — copied unchanged from Doc04: the raw version the chain is compared against.

For this document, save your practice code as:

- **Basic** (your first LCEL chain) is its own topic — save it as `practice/lcel_chain_basics_practice.py`.
- **Intermediate** (swap in a structured parser) and **Failure** (force the parser to actually fail) are both about structured-output parsing and its errors — save them together as `practice/structured_output_practice.py`, one section per level.
- **Real-world** (rebuild Doc04's extraction, LCEL-style) is its own topic — save it as `practice/lcel_vs_raw_sdk_practice.py`.
- **Real-world (part 2)** (run in parallel, fall back, and stream) is its own topic — save it as `practice/lcel_patterns_practice.py`.
- **Real-world** (a support-reply router) is its own topic — save it as `practice/runnable_routing_practice.py`.
- **Edge cases** (a template with the wrong variables) is its own topic — save it as `practice/prompt_template_practice.py`.

Why group by topic instead of by level: if you save each exercise by difficulty level instead, the different versions of the same idea end up scattered across separate files, and you can never see how one topic grows from simple to harder in one place. Grouping by topic keeps that growth visible — open one file, and you see the whole journey for that one thing, from basic to advanced, side by side.

Jump to an exercise: Basic · Intermediate · Real-world · Real-world (part 2) · Real-world (router) · Edge cases · Failure · Build Task

Basic — your first LCEL chain

- **What:** connect a prompt template → `ChatOpenAI` → text output parser with `|`.
- **Why:** this 3-piece chain is the atom every LangChain thing you'll ever build is made of — get comfortable with it before adding anything else.
- **How to code it:** `prompt = ChatPromptTemplate.from_template("Answer: {question}")`, `chain = prompt | ChatOpenAI() | StrOutputParser()`, then `chain.invoke({"question": "..."})`. Then wrap it in `build_chain(config: Config | None = None)`, which takes the key from Doc01's `Config` — loaded when none is passed.
- **Save as:** `practice/lcel_chain_basics_practice.py`.

- **Builds on:** Doc04's Basic exercise (`practice/chat_api_basics_practice.py`) — the same first call, now split into a prompt, a model, and a parser instead of one raw `client.chat.completions.create(...)`.
- **Used later by:** the Real-world exercise below and the Build Task — this exact three-piece shape, reused.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate — swap in a structured parser

- **What:** replace the text parser with a structured one (`.with_structured_output(Person)`), and see what comes back when the text has no answer in it.
- **Why:** this is Doc04's "shape during" method reached through LangChain — including its trap: a required field makes the model invent a value instead of raising an error.
- **How to code it:** define `Person` (`name`, `age`), run it on a clear input, then on "the sky is blue" — read what comes back. Then make both fields `| None` and write `extract_person(text) -> Person | None`.
- **Save as:** `practice/structured_output_practice.py`, under an `# Intermediate` section (this file also holds the Failure exercise below, in its own `# Failure` section).
- **Builds on:** the Basic exercise's chain, and Doc04's structured-output topic — `.with_structured_output(Model)` is the exact same constrained-decoding call, just reached through LangChain.
- **Used later by:** the Failure exercise right below (same file), and the Build Task's parser choice.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — rebuild Doc04's extraction, LCEL-style

- **What:** take Doc04's Build Task extraction (`get_structured_reply()` + `ExtractedInfo`) and rebuild it with LCEL, keeping both versions.
- **Why:** this is the only way to actually compare LangChain vs. raw SDK — reading about the trade-off isn't the same as having both versions of the same feature side by side.
- **How to code it:** copy Doc04's `chat_client.py` and `schemas.py` into `practice/` unchanged. Build `ChatPromptTemplate.from_messages([("system", EXTRACT_PROMPT), ("human", "{text}")]) | model.with_structured_output(ExtractedInfo)`, run both versions on the same 3 inputs, and check the names match.
- **Save as:** `practice/lcel_vs_raw_sdk_practice.py`.
- **Builds on:** Doc04's Build Task extraction (the raw-SDK version), the Basic exercise's chain shape, and the Intermediate exercise's `.with_structured_output()`.
- **Used later by:** the Build Task's `compare_with_raw_sdk.py`, which **copies** this same comparison almost unchanged.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world (part 2) — run in parallel, fall back, and stream

- **What:** run two slow lookups at the same time with `RunnableParallel`, add a backup with `.with_fallbacks()` + `RunnableLambda`, and stream a reply with `.stream()` — then prove each one works.
- **Why:** the LCEL topic says every chain can do these three things for free; doing each once is what makes them usable, not just readable.
- **How to code it:** two functions that `time.sleep(1)`, timed with `time.time()`; a model pointed at `base_url="http://127.0.0.1:9"` (nothing listens there) for a real outage; `exceptions_to_handle=` so a bug in your own code still raises; and a loop over `chain.stream(...)` that joins the pieces.
- **Save as:** `practice/lcel_patterns_practice.py`.
- **Builds on:** the Basic exercise's `prompt | model | parser`, and Doc04's streaming exercise and error table.
- **Used later by:** Project 6 — its lookup step, its fallbacks and its streamed replies are these exact shapes.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — a support-reply router

- **What:** build a support desk that routes a ticket to the right reply chain by its category — `billing`, `technical`, or anything else — using `RunnableBranch`, then tags the reply with a `RunnableLambda`. A real routing pipeline, not a toy.
- **Why:** this is the shape of almost every production LCEL pipeline (and Project 6): classify the input, send it to the matching chain, post-process the result. It ties together the three chain shapes (sequential `|`, branch, and your own function) in one meaningful task.
- **How to code it:** three reply chains (`ChatPromptTemplate.from_messages([("system", ...), ("human", "{text}")]) | model | StrOutputParser()`), one per category — billing, technical, and a general default. Wrap them in `RunnableBranch((cond, chain), ..., default_chain)` keyed on `ticket["category"]`. Add a `RunnableLambda` final step that appends `"(auto-routed)"`. Then run one ticket per category — **including an unknown one** — and confirm the right branch (and the default) runs each time.
- **Save as:** `practice/runnable_routing_practice.py`.
- **Builds on:** the Basic exercise's `prompt | model | parser`, Step 6's `from_messages`, and Steps 8-10's `RunnableLambda` / `RunnableBranch`.
- **Used later by:** Project 6's ticket-triage pipeline, which routes exactly this way, and Doc11's Supervisor, which is this routing idea one layer up.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Edge cases — a template with the wrong variables

- **What:** a prompt template missing a variable it needs, and one given an extra variable it doesn't use — see what fails, and exactly when.
- **Why:** knowing whether a broken template fails *while building the prompt* or *only once the API call goes out* changes how fast you can debug it.
- **How to code it:** call `.invoke({})` on a template expecting `{question}` — read the exact error. Then call it with `{"question": "...", "extra": "..."}` and confirm whether it's silently ignored or errors.
- **Save as:** `practice/prompt_template_practice.py`.
- **Builds on:** the Basic exercise's prompt template — same template, now testing its failure modes instead of the happy path.
- **Used later by:** the Build Task's `prompts.py`, which must fail the same clear way.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Failure — force the parser to actually fail

- **What:** use a "check after" parser (`PydanticOutputParser`) with a prompt that never mentions the shape, and read the real error it raises. Then fix the prompt with the parser's format instructions.
- **Why:** you need to know, before it happens in production, exactly what kind of exception your code needs to catch here.
- **How to code it:** `prompt | model | PydanticOutputParser(pydantic_object=Rating)` with the prompt "Rate {thing} from 1 to 10." — catch `OutputParserException` by name. Then add `{format_instructions}`, fill it with `.partial()`, and test `parser.parse(...)` on plain text with no API call.
- **Save as:** `practice/structured_output_practice.py`, under a `# Failure` section (this file also holds the Intermediate exercise above, in its own `# Intermediate` section).
- **Builds on:** the Intermediate section of this same file ("shape during" — this is "check after"), the Edge cases exercise's templates, and Doc04's error-handling habit — catch the specific parser exception, never a bare `except`.
- **Used later by:** the Build Task's "wrong shape" test case (`parser.parse(...)` → `OutputParserException`).
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Build Task — Reusable Chain Module

Stuck on the Build Task? Hint 1 · Hint 2 · Show me the solution

Goal: a small chain module that later documents will reuse instead of writing raw SDK calls each time — a comparison point next to Doc04's raw-SDK version, not a replacement for it.

Requirements:

- One LCEL chain that wraps a prompt template, a model, and a parser as one callable piece.
- The model choice (name, temperature) lives in one named place — `MODEL_NAME` and the default parameters of `build_extraction_chain()`, the Basic exercise's shape — never scattered. Doc01's `config.py` / `get_logger(name)` are copied in unchanged, the way Doc04's Build Task did (Doc01's `Config` holds only the key and the log level).
- A test script that proves your LCEL version and Doc04's raw-SDK version agree for the same inputs, and checks the two failure cases.

Inputs: the same structured-extraction task as Doc04's Build Task (`/extract`).

Outputs: the same typed `ExtractedInfo` as Doc04's, but built a different way.

Constraints: don't delete or replace the Doc04 raw version — you need both, to compare them and to answer the interview question "when would you use which."

```
05_langchain_fundamentals/practice/build_task/
├── chain.py              build_extraction_chain() -> Runnable
├── prompts.py            get_extraction_prompt()
├── compare_with_raw_sdk.py  the 3 Test Cases
├── chat_client.py        copied from 04_openai_api's Build Task
├── schemas.py            copied from 04_openai_api's Build Task
├── config.py             copied from 01_python_foundations's Build Task
├── exceptions.py         copied from 01_python_foundations's Build Task
└── logging_setup.py      copied from 01_python_foundations's Build Task
```

- `chain.py` / `prompts.py` — **What/Why:** the one chain module every later document imports instead of writing raw SDK calls again.
- `compare_with_raw_sdk.py` — **What/Why:** this Build Task's test file — proves the LCEL version and Doc04's raw version agree on every name, and that a missing slot and a wrong shape fail with named errors.
- `chat_client.py` / `schemas.py` — **What/Why:** copied unchanged from Doc04 — the raw version the chain is compared against.
- `config.py` / `exceptions.py` / `logging_setup.py` — **What/Why:** copied unchanged from Doc01, so this module uses the exact same settings/logging behavior as every other document.

Run it: `cd practice/build_task && python compare_with_raw_sdk.py` — from inside the folder, so `from config import load_config` finds the file next to it.

Builds on: Doc01's Build Task `config.py` / `get_logger(name)` — **copy** both in unchanged, same as Doc04 did. Doc04's Build Task extraction (`chat_client.py` , `schemas.py`) — copied unchanged, the same task rebuilt with LCEL. All five exercises above — the Basic `build_chain()` shape, the Intermediate `.with_structured_output()` , the Real-world comparison, the Edge cases `KeyError` , and the Failure `OutputParserException` — fold into this one module.

Used later by: Doc06 attaches tools to this same `ChatOpenAI` object with `.bind_tools(...)`; Doc08 reuses this LCEL chain shape for its RAG pipeline; Project 6 builds directly on this chain, adding retries and fallbacks on top.

Functions/Components to build:

- `prompts.py` → `get_extraction_prompt() -> ChatPromptTemplate`
- `chain.py` → `build_extraction_chain(model_name, temperature) -> Runnable`
- `compare_with_raw_sdk.py` → `run_comparison()` and one test function per Test Case

Expected Behavior

- The chain finds the same name as Doc04's raw version, for the same input (the intent's wording can differ between any two calls).
- Changing the prompt template's variables shouldn't require touching the chain-building code.

Test Cases

Scenario	Expected
A well-formed input	Chain and raw-SDK results agree on the name
A missing template variable	A clear error while building the prompt, not silently left blank
A reply that doesn't fit the shape	`OutputParserException` — a clear, named, catchable error

Break-It / Debug Preview

- A chain that quietly returns the wrong Python type, because the parser doesn't actually match what the prompt asks for.
- Full debugging drill in 14_debugging_lab.

Interview Topics Preview

- When LangChain is worth its cost vs. when the raw SDK is simpler and easier to debug · what LCEL actually connects when it runs · `langchain-core` vs. `langchain-openai` vs. `langchain`.

Move On When

You can decide, for a given task, LangChain vs. raw SDK — not just default to whichever you learned last. Full details: CURRICULUM.md §4.

Stuck? Ask for **Hint 1** or **Hint 2** (each has Basic/Intermediate depth). Ask for the full solution only if you say **"Show me the solution."**